TP : Infrastructure Discord

Phase 0 : Lire

Information de documentation :

1. Discord a dû faire face à une croissance considérable du volume de messages. En effet, le nombre est passé de 40 millions à plus de 120 millions de messages par jour en 2016.

2. Ils avaient initialement un système de stockage basé sur MongoDB, qui est devenu inadapté lorsque les données ont dépassé la capacité de la mémoire vive, entraînant des temps de réponse imprévisibles.

3. Discord a ensuite opté pour Cassandra afin de répartir les messages sur plusieurs serveurs, en utilisant une clé de partition qui combine l'identifiant du salon et une fenêtre temporelle.

4. Cependant, cette solutions a rencontré plusieurs défis : accumulation de marqueurs de suppression, partitions trop sollicitées, ralentissements et pauses dues au ramasse-miettes de la JVM.

5. Pour surmonter ces problèmes, Discord a migré vers Scylladb et a développé des services de données en Rust pour améliorer les performances, réduire la latence et mieux gérer les requêtes.

Trois chiffres clés :

1. Plus de 120 millions de messages par jour Ce chiffre illustre le volume quotidien traité par Discord en 2016. Il met en lumière l'ampleur des données et explique pourquoi une architecture classique avec un stockage centralisé devient limitée. Il est donc essentiel d'imaginer une infrastructure capable d'accueillir une croissance significative des messages qui pourrait encore augmenter à l'avenir.

2. La limite maximale d'une partition Cassandra qui est fixée à 100 Mo Lorsque cette limite était atteinte ou dépassée, Cassandra émettait des avertissements dans les journaux. Discord a donc choisi de segmenter les messages en fenêtres temporelles d'environ dix jours pour éviter que les partitions ne deviennent trop lourdes. Cette donnée influence directement le choix de la clé de partition ainsi que l'organisation des informations.

3. Les performances en lecture : 15 ms contre 40 à 125 ms Après le passage à Scylladb, la latence pour lire l'historique des messages au niveau du 99e centile est passée d'environ 40 à 125 ms avec Cassandra à environ 15 ms avec Scylladb. Cela démontre

que le choix du moteur de stockage joue un rôle crucial pour atteindre des performances adaptées aux attentes des utilisateurs, surtout lorsqu'ils consultent fréquemment les derniers messages.

Phase 1 : Cadrer le besoin et modéliser

1a : Liste des acteurs, objets métier et cycle de vie

Acteurs :

Acteur	Description	Actions principales
Utilisateur	Personne qui a un compte sur la plateforme.	Créer ou se connecter avec un compte, rejoindre un serveur, envoyer des messages, ajouter des amis.
Propriétaire	Utilisateur qui a créé ou possédé un serveur.	Gérer son serveur, ses membres et ses salons.
Modérateur	Membre disposant de droits de gestion sur un serveur.	Modérer les échanges et gérer les membres selon ses permissions.
Membre	Utilisateur qui appartient à un serveur.	Consulter les salons et envoyer des messages dans les salons.
Système	Partie d'automatisation de la plateforme.	Gérer les présences, les statistiques et les suggestions que ce soit pour le salon ou les membres.

Objets métiers :

Objet	Création	Modification	Suppression / expiration
Utilisateur	Inscription sur la plateforme.	Modification du pseudo, du courriel ou de son mot de passe.	Suppression du compte, selon une politique à définir.
Serveur	Création par un utilisateur.	Modification de ses informations.	Suppression par son utilisateur propriétaire.
Appartenance	Lorsqu'un utilisateur rejoint un serveur.	Modification du rôle (membre, modérateur, propriétaire).	Arrivée ou exclusion du membre.

Objet	Création	Modification	Suppression / expiration
Salon	Création par un responsable du serveur.	Modification du nom ou du type.	Suppression par une personne qui a l'autorisation.
Message	Envoi dans un salon textuel.	Modification du contenu comme le texte.	Suppression par son auteur ou un modérateur autorisé.
Pièce jointe	Ajouter avec un message.	Modification de ses informations ou remplacement.	Suppression avec le message ou indépendamment, selon la règle retenue.
Amitié	Demande ou création d'une relation entre deux utilisateurs.	Éventuellement gestion de son état.	Suppression de la relation par l'un des utilisateurs.
Présence	Connexion ou signal de vie d'un utilisateur.	Actualisation régulière.	Expiration après 5 minutes environ sans activité.
Statistiques	Calcul quotidien automatique.	Actualisation quotidienne.	Conservation selon une durée à définir.

Glossaire :

Terme	Définition
Utilisateur	Personne qui a un compte unique sur la plateforme.
Serveur	Espace de communauté qui regroupe des utilisateurs et salons.
Membre	Personne appartenant à un serveur particulier.
Propriétaire	Membre possédant le rôle de gestion principal d'un serveur.
Modérateur	Membre qui a des permissions supplémentaires pour gérer le serveur.
salon	Espace de discussion appartenant à un serveur. Il peut être textuel ou vocal.

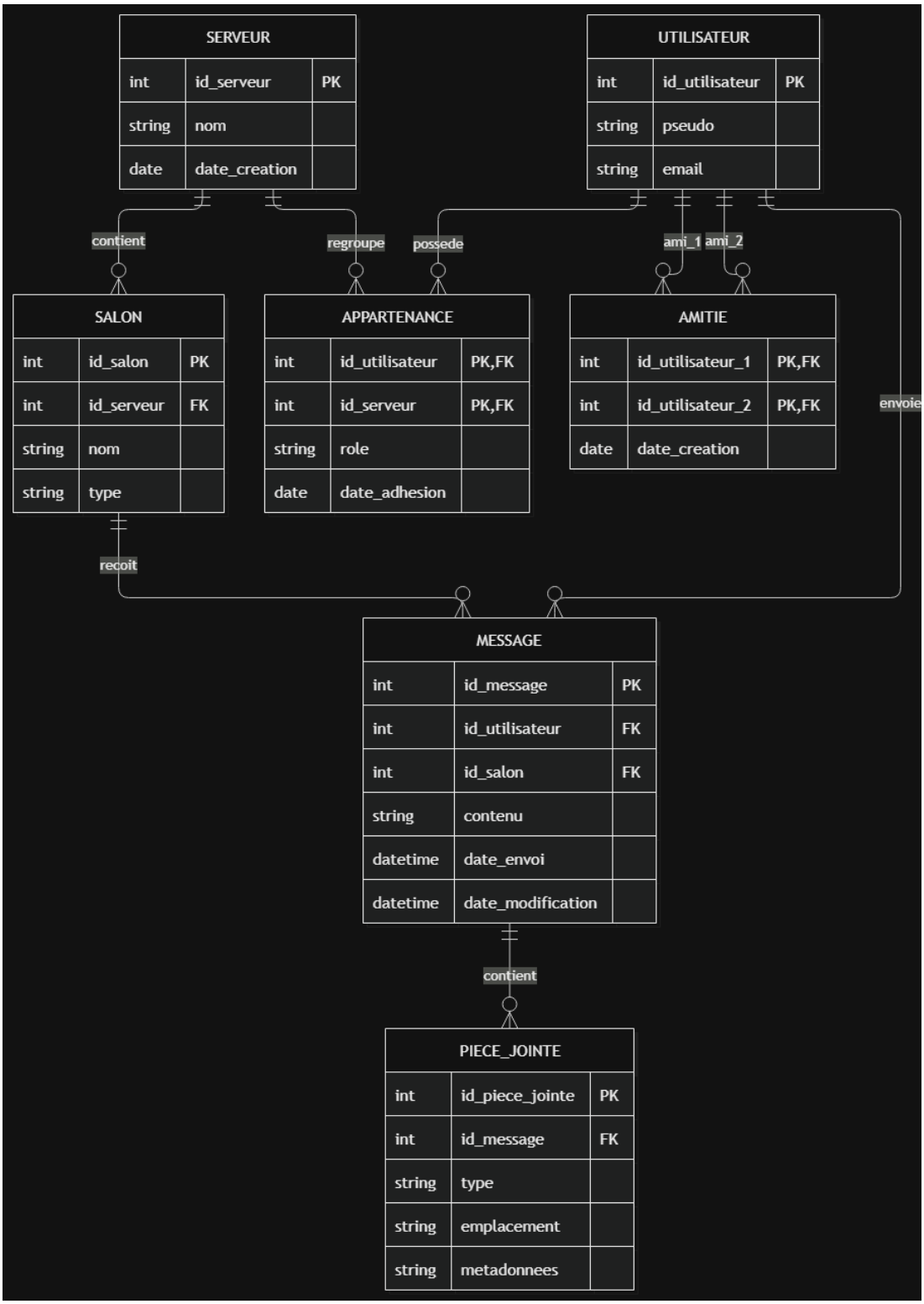
Terme	Définition
Message	Texte envoyé par un utilisateur dans un salon textuel.
Pièce jointe	Élément associé à un message, ça peut être une image, fichier ou lien.
Amitié	Relation entre deux utilisateurs de la plateforme.
présence	État indiquant si un utilisateur est actuellement en ligne ou absent.

1b : Règles de gestion

Garantie par le modèle	Garantie seulement par du code ou un trigger	Hors modèle relationnel
Clé primaire sur l'identifiant utilisateur.	Garantie qu'un serveur possède toujours un propriétaire.	Redis ou stockage en mémoire. Présences fréquentes et temporaires.
Contrainte « unique » sur l'adresse électronique.	Vérifie que le propriétaire appartient bien au serveur.	Redis avec temps de 300 secondes pour suppression automatique.
Table associative entre utilisateur et serveur.	Empêche qu'un serveur possède plusieurs propriétaires.	Graphe ou requêtes relationnelles pour parcourir les liens amis/serveurs.
Attribut rôle dans la table d'appartenance.	Vérifie que le salon associé au message est bien de type textuel.	Graphe ou requêtes relationnelles pour explorer le réseau d'amis.
Contrainte « check » sur les valeurs autorisées du rôle.	Vérifie que l'auteur du message appartient au serveur du salon.	Table d'agrégats ou moteur analytique pour éviter les recalculs.
Clé étrangère entre salon et serveur.	Déclenche automatiquement l'expiration d'une présence après 5 min.	

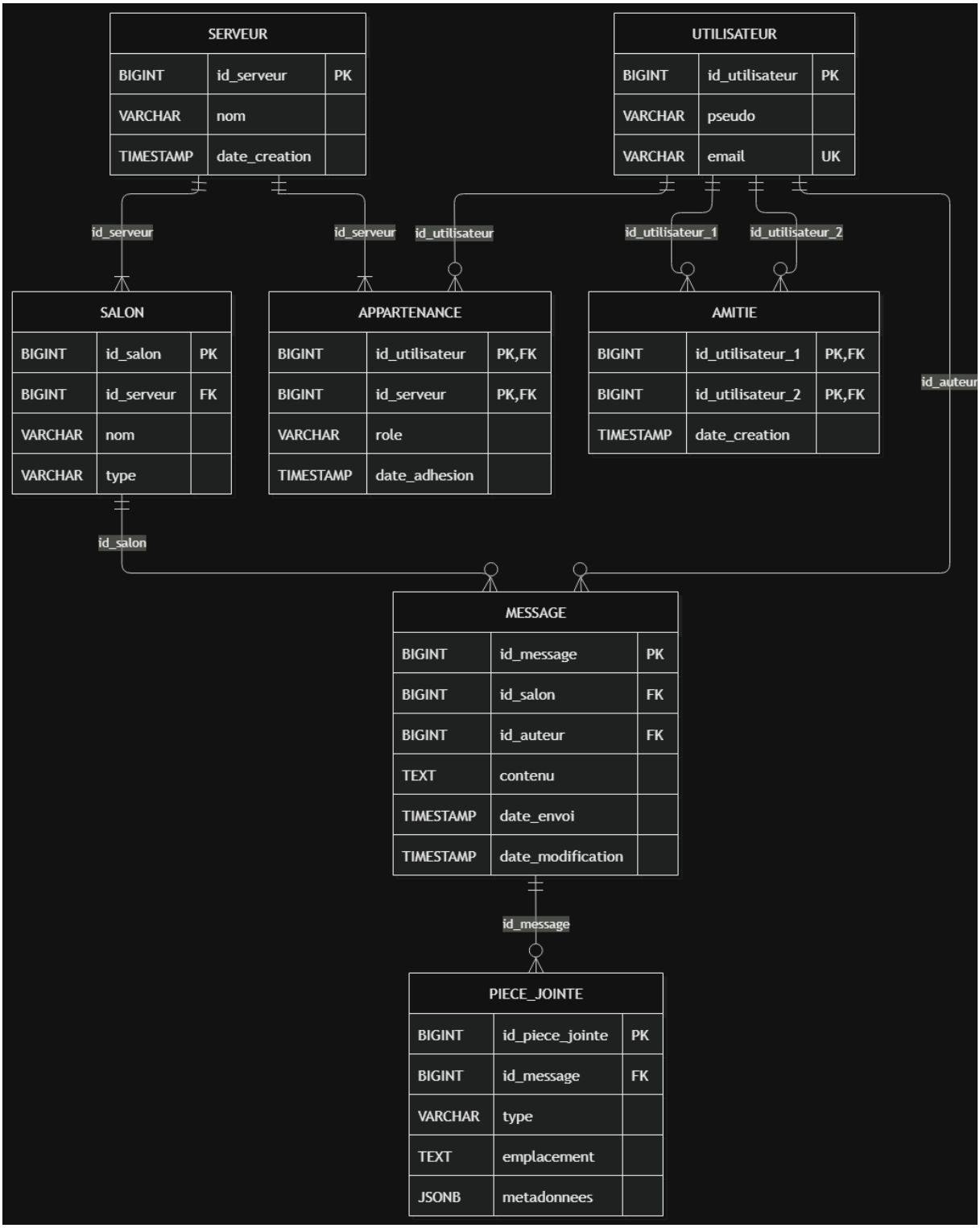
Clé étrangère « server_id » dans la table salon.	Parcourir les relations d'amitié et les appartenances aux serveurs.	
Contrainte « Unique » sur (server_id, name).	Parcourir les relations entre amis pour identifier de nouvelles personnes.	
Contrainte « Check » sur le type de salon.	Calculer quotidiennement les statistiques des messages.	
Clé étrangère entre message et salon.	Exécution d'une requête récupérant les 50 derniers messages.	
Colonnes « Not Null » (auteur, contenu, date).		
Colonne nullable « edited_at ».		
Clé étrangère entre pièce jointe et message.		
Contrainte « Check » sur le type de pièce jointe.		
Clé primaire composée et ordre canonique des deux Id utilisateurs.		

1c : MCD

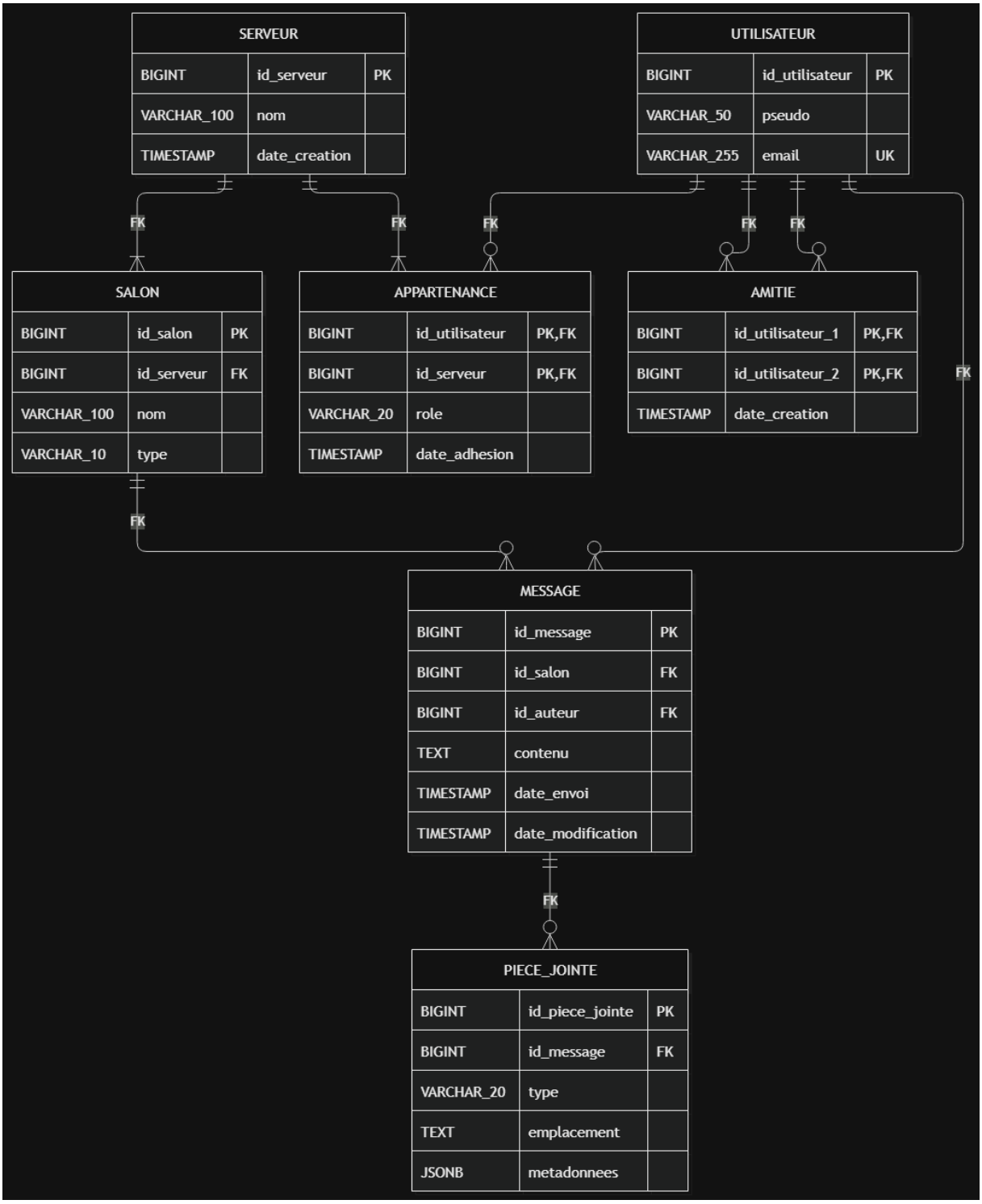

SERVEUR
int id_serveur PK
string nom
date date_creation
UTILISATEUR
int id_utilisateur PK
string pseudo
string email
contient
regroupe
possede
ami_1
ami_2
envoie
SALON
int id_salon PK
int id_serveur FK
string nom
string type
APPARTENANCE
int id_utilisateur PK,FK
int id_serveur PK,FK
string role
date date_adhesion
AMITIE
int id_utilisateur_1 PK,FK
int id_utilisateur_2 PK,FK
date date_creation
recoit
MESSAGE
int id_message PK
int id_utilisateur FK
int id_salon FK
string contenu
datetime date_envoi
datetime date_modification
contient
PIECE_JOINTE
int id_piece_jointe PK
int id_message FK
string type
string emplacement
string metadonnees

1d : MLD et MPD

MLD :



MPD :


SERVEUR
BIGINT id_serveur PK
VARCHAR_100 nom
TIMESTAMP date_creation
UTILISATEUR
BIGINT id_utilisateur PK
VARCHAR_50 pseudo
VARCHAR_255 email UK
SALON
BIGINT id_salon PK
BIGINT id_serveur FK
VARCHAR_100 nom
VARCHAR_10 type
APPARTENANCE
BIGINT id_utilisateur PK,FK
BIGINT id_serveur PK,FK
VARCHAR_20 role
TIMESTAMP date_adhesion
AMITIE
BIGINT id_utilisateur_1 PK,FK
BIGINT id_utilisateur_2 PK,FK
TIMESTAMP date_creation
MESSAGE
BIGINT id_message PK
BIGINT id_salon FK
BIGINT id_auteur FK
TEXT contenu
TIMESTAMP date_envoi
TIMESTAMP date_modification
PIECE_JOINTE
BIGINT id_piece_jointe PK
BIGINT id_message FK
VARCHAR_20 type
TEXT emplacement
JSONB metadonnees
FK

1e : questions au client :

Hypothèse 1 : Un utilisateur peut-il posséder plusieurs serveurs ?

Question à poser au client :

Un même utilisateur peut-il être membre de plusieurs serveurs ?

Conséquence sur le modèle selon la réponse :

Si oui, alors la relation entre « utilisateur » et « Serveur » reste une relation de type (N, N), représentée par l'entité associative « appartenance ».

Si non, alors une contrainte en plus devra être ajoutée afin de limiter chaque utilisateur à un seul serveur.

Hypothèse 2 : Un serveur peut-il posséder plusieurs propriétaires ou plusieurs modérateurs ?

Question à poser au client :

Un serveur peut-il avoir plusieurs propriétaires et plusieurs modérateurs, ou chaque rôle doit-il être attribué à une seule personne ?

Conséquence sur le modèle selon la réponse :

- Si plusieurs propriétaires sont autorisés : la contrainte d'unicité partielle devra être modifiée afin de permettre plusieurs propriétaires pour un même serveur.
- Si un seul propriétaire est autorisé : la contrainte d'unicité partielle actuelle sera conservée afin de garantir qu'un serveur ne possède qu'un seul propriétaire.

Hypothèse 3 : Un message peut-il contenir plusieurs pièces jointes ?

Question à poser au client :

Un utilisateur peut-il ajouter plusieurs pièces jointes (images, fichiers, documents, etc.) à un même message ?

Conséquence sur le modèle selon la réponse :

Si oui, alors la relation entre « Message » et « piece_jointe » reste de type (1, N), permettant d'associer plusieurs pièces jointes à un même message.

Si non, alors une contrainte « Unique » devra être ajoutée sur l'attribut id_message dans « piece_jointe » afin de limiter chaque message à une seule pièce jointe.

Hypothèse 4 : Les messages supprimés doivent-ils être conservés pour l'historique ou définitivement effacés ?

Question à poser au client :

Lorsqu'un utilisateur supprime un message, celui-ci doit-il être conservé dans la base de données pour garder une trace de l'historique, ou doit-il être définitivement supprimé ?

Conséquence sur le modèle selon la réponse :

Si les messages doivent être conservés : une suppression logique devra être mise en place, avec l'ajout d'un attribut « deleted_at » dans l'entité « message » pour enregistrer la date de suppression.

Si les messages doivent être définitivement effacés : une suppression physique pourra être effectuée directement dans la base de données.

Hypothèse 5 : Les utilisateurs peuvent-ils consulter les messages d'un serveur après l'avoir quitté ?

Question à poser au client :

Un utilisateur qui quitte un serveur peut-il continuer à consulter les anciens messages de ce serveur ?

Conséquence sur le modèle selon la réponse :

Si oui, les règles de gestion des accès devront distinguer l'appartenance actuelle au serveur des droits de consultation de l'historique des messages.

Si non, l'accès aux messages devra être limité aux utilisateurs qui sont actuellement membres du serveur.

Phase 2 : Qualifier la charge

2a : Table d’accès

Accès	Lecture ou écriture	Fréquence	Latence visée	Forme de l'accès	Cohérence exigée
Envoyer un message	Écriture	Très élevée	Environ 50 ms	Écriture par clé de salon	Stricte
Afficher les 50 derniers messages	Lecture	Très élevée, accès primordial	Environ 50 ms	Recherche par salon et plage de messages récents	Stricte
Vérifier le droit d'écrire	Lecture	Très élevée	Environ 20 ms	Recherche par utilisateur et serveur	Stricte
Afficher les utilisateurs en ligne	Lecture	Très élevée	Environ 50 ms	Lecture par serveur ou utilisateur	Éphémère
Actualiser la présence d'un utilisateur	Écriture	Élevée	Environ 20 ms	Écriture par identifiant utilisateur	Éphémère
Suggérer des serveurs	Lecture	Moyenne	Environ 200 ms	Parcours des relations d'amitié et des appartenances	Tolérance au retard
Suggérer des amis	Lecture	Moyenne	Environ 200 ms	Parcours des relations d'amitié	Tolérance au retard
Calculer l'activité quotidienne d'un serveur	Lecture ou agrégat	Une fois par jour	3 secondes	Agrégat par serveur et par date	Tolérance au retard
Supprimer un message	Écriture	Faible à moyenne	Environ 100 ms	Recherche par identifiant de message	Stricte

Accès	Lecture ou écriture	Fréquence	Latence visée	Forme de l'accès	Cohérence exigée
Rechercher dans l'historique	Lecture	Moyenne	Environ 200 ms	Recherche dans le contenu des messages	Tolérance pour le retard

2b : Volumétrie

1. Nombre moyen de messages écrits par seconde en décembre 2016, puis au pic :

Info : 100 millions de messages par jour en décembre 2016

Journée en seconde :

24 × 60 × 60 = 86 400 secondes

Moyenne : 100 000 000 ÷ 86 400 = 1 157,4 messages par seconde

On va prendre 1 157 messages par secondes.

Calcul du pic :

1 157,4 × 3 = 3 472,2 messages par seconde.

Il faut que le système doit pouvoir supporter 3472 messages par secondes pendant les fortes activités.

2. Volume de messages stockés par jour et par an :

Information :

Un message stocké vaut environ 300 octets

Discord traite 120 millions de messages par jour

120 000 000 × 300 = 36 000 000 000 octets.

Cela représente environ 36 Go par jour à gérer pour Discord.

36 * 365 = 13 140 Go soit 13, 14 To

Le volume devient vite important avec 13.14 To, il faut donc une solution pour surveiller la croissance et gérer le stockage.

3. Taille de la table de messages et de l'index à 100 millions de messages :

Info suivante :

MongoDB avant crash : 16 Go

Message : 300 octets

Entrée index : 40 octets

Calcul taille données :

100 000 000 × 300 = 30 000 000 000 octets.

Calcul de la taille de l'index :

100 000 000 × 40 = 4 milliards octets.

Taille totale : 30 + 4 = 34 Go

Donc pour 34 Go sachant que MongoDB peut tenir qu'a 16 Go. Les données et l'index ne peuvent plus tenir simultanément en mémoire vive. Les requêtes peuvent devenir longues et les performances se dégrade. Ce qui explique pourquoi Discord rencontre des difficultés avec MongoDB.

4. Contrôle de vraisemblance

On sait que Cassandra avec ses nœuds en a 12, un facteur de réplication de 3 et 1 To de données compressées par nœud.

On fait le calcul total du volume physique

12 × 1 = 12 To.

On a en réplication 3, donc on le divise le volume physique au total par 3.

12 / 3 = 4 To

Le cluster représente donc environ 4 To de données qui sont compressés.

4 000 000 000 000 ÷ 300 = 13,33 milliards de messages.

Donc le volume logique est de 13.33 milliards de message. On peut voir que cela est cohérent par rapport aux dossiers de Discord.

2c : Dénormaliser ou pas ?

Cas 1 (le plus probable) : Afficher les 50 ou 100 derniers messages d'un salon

Contrainte : Cette requête est la plus fréquente de toute la plateforme. Elle doit être rapide et récupérer uniquement les messages récents d'un salon.

Technique de dénormalisation : On pourrait créer une structure de stockage spécialement organisée par salon, avec les messages classés par identifiant chronologique.

On peut aussi conserver les derniers messages dans un cache afin d'éviter de relire systématiquement la base principale.

Coût :

- Les messages doivent être écrits dans plusieurs emplacements.
- Le cache doit être actualisé après chaque nouveau message.

Garde-fou : La base principale reste la source de vérité. Le cache peut être reconstruit si nécessaire et doit avoir une durée de vie limitée.

Cas 2 : Montrer les utilisateurs en ligne

Contrainte : la présence des utilisateurs doit être affichée en permanence. Une vérification répétée dans une base relationnelle pourrait générer beaucoup de requêtes.

Technique de dénormalisation :

On utilise une structure de type clé valeur pour stocker la présence, on aura « Clé » qui sera identifiant utilisateur. La « Valeur » qui est le statut et dernière activité. Et aussi une expiration automatique après cinq minutes si l'utilisateur est absent.

Cela permet de retrouver rapidement le statut d'un utilisateur au lieu de parcourir plusieurs tables.

Coût :

- Les données de présence sont temporaires.
- Elles peuvent être perdues si le système de stockage rencontre un problème.
- Elles doivent être régulièrement actualisées.

Garde-fou : On met en place une expiration automatique de cinq minutes et on considère la présence comme une donnée temporaire, qui est reconstruite lorsque l'utilisateur se reconnecte.

Cas 3 : Calculer les statistiques quotidiennes d'un serveur

Contrainte : le tableau de bord doit afficher le nombre de messages et le nombre d'auteurs distincts par jour.

Technique de dénormalisation : On va créer une table de statistiques précalculées contenant l'identifiant du serveur, la date, le nombre de messages et le nombre d'auteurs distincts. On mettra une tâche quotidienne qui met à jour cette table.

Coût :

- Les statistiques ne sont pas instantanées.
- Les résultats peuvent être légèrement en retard.
- Une erreur ou exception dans le calcul peut produire des résultats incorrects.

Garde-fou : On conserve les messages comme source de vérité et on recalcule régulièrement les statistiques pour vérifier leur cohérence.

L’accès que je refuse de dénormaliser est de vérifier qu'un utilisateur a le droit d'écrire dans un salon, car Il serait risqué de conserver uniquement une copie de cette information dans une structure secondaire, parce qu’une modification des appartenances pourrait ne pas être immédiatement répercutée. Un utilisateur qui vient d'être retiré d'un serveur pourrait alors continuer à envoyer des messages.

Il serait mieux d’effectuer cette vérification à partir des données de référence.

Phase 3 : Reconstituer la décision de stockage des messages

3a : Autopsie de 2015

C’est parce que le système possède donc environ deux fois plus de données que sa mémoire ne peut en contenir. En conséquence, les données et l'index ne tiennent plus dans la mémoire vive du serveur. Lorsqu'un utilisateur demande les 50 derniers messages d'un salon, MongoDB utilise l'index pour retrouver les documents correspondants. Cependant, lorsque les données dépassent la mémoire disponible, MongoDB doit régulièrement accéder au disque pour récupérer les documents. Les lectures deviennent alors plus lentes et les performances deviennent donc imprévisibles, car certaines requêtes trouvent leurs données en mémoire tandis que d'autres doivent les récupérer sur le disque, ce qui prend beaucoup plus de temps.

De plus, même si l'index reste en mémoire vive, cela ne garantit pas une lecture rapide. L'index permet de localiser les documents, mais les documents eux-mêmes doivent encore être récupérés. Si les données ne sont pas présentes en mémoire, le serveur doit effectuer des accès disque. Cela provoque une augmentation de la latence et peut rendre les temps de réponse irréguliers.

Après, pour savoir pourquoi le profil « serveur à dominante vocale » est-il le pire cas. C’est parce qu’il génère très peu de messages. Lorsqu'un utilisateur demande les 50 derniers messages, le système doit parfois remonter très loin dans l'historique. Les données recherchées sont donc rarement présentes dans le cache.

Or sur la documentation de Discord, si on regarde le serveur public, a l'inverse, un grand serveur public génère beaucoup de messages récents. Les données les plus consultées restent souvent en mémoire.

3b : Qui prétend au rôle ?

Pour comparer les quatre en fonction des critères, on va plutôt faire un tableau :

Critères	PostgreSQL	MongoDB	Cassandra	Redis
Modèle de données	Relationnel, tables et relations	Documents Json ou Bson	Colonnes larges, données organisées en partitions	Clé valeur et structures de données
Stock des messages	Une table « message » avec	Une collection de documents	Une partition par salon et	Une liste ou structure triée par salon

Critères	PostgreSQL	MongoDB	Cassandra	Redis
	un identifiant de salon		fenêtre temporelle	
Récupération des 50 derniers messages	Requête SQL avec filtre et tri	Requête avec filtre et tri sur les champs qui sont indexés	Lecture d'une partition et récupération des messages	Lecture rapide des dernières entrées
Lorsque les données dépassent la RAM	Accès au disque et cache	Accès au disque et cache	Lecture depuis les SS Tables et les structures mémoire	Nécessite suffisamment de mémoire
Passage à l'échelle horizontal	Possible, mais cela nécessite une architecture supplémentaire	Possible avec seulement le sharding	Conçu pour la distribution en horizontale	Possible avec Redis Cluster
Transactions	Transactions relationnelles complètes	Transactions disponibles	Transactions limitées aux opérations prévues par le modèle	Opérations atomiques, mais pas de jointures relationnelles classiques
Jointures	Très bonnes capacités	Pas de jointures relationnelles classiques mais agrégations possibles	Pas de jointures classiques	Pas de jointures
Requêtes ad hoc	Très bonnes capacités sur les requêtes SQL	Requêtes flexibles sur les documents	Requêtes limitées par la clé de partition	Très limitées en mémoire

Si on devrait conclure, pour cette requête précise, Cassandra est particulièrement intéressante lorsque le volume de messages devient très important. Elle permet de répartir les données sur plusieurs nœuds de serveur et d'organiser les messages par salon et par fenêtre temporelle.

Pour PostgreSQL, il reste intéressant pour les données relationnelles et les requêtes nécessitant des jointures.

Ensuite, MongoDB peut convenir à une organisation documentaire, mais l'expérience de Discord montre ses limites lorsque les données et les index dépassent la mémoire disponible.

Et pour finir, Redis est très performant pour les accès rapides, mais conserver tout l'historique des messages en mémoire serait coûteux en performances.

3c : Concevoir la clé de partition.

Calculer la taille des partitions pour un salon de 10 millions de messages par an :

Information :

Un salon contient 10 millions de messages par an.

Calcul de la taille annuelle :

10 000 000 × 300 = 3 000 000 000 octets soit 3 Go par an

Option A : une partition par salon

Taille de la partition : 3 Go.

Cette option dépasse la limite de 100 Mo.

Option B : une partition par salon et par fenêtre de 10 jours

Nombre moyen de messages sur 10 jours :

10 000 000 / 365 × 10 = 273 973 messages.

Taille de la partition :

273 973 × 300 = 82 191 900 octets.

Soit 82,2 Mo.

Option C : une partition par salon et par jour

Nombre moyen de messages par jour :

10 000 000 / 365 = 27 397 messages.

Taille de la partition :

27 397 × 300 = 8 219 100 octets.

Soit 8,2 Mo.

Donc, la meilleure solution est la C, mais la B est une alternantive.

Pour un salon de 1 000 messages par an, combien de partitions faut-il interroger pour obtenir 50 messages :

Information : 1000 messages par an

Option B : fenêtre de 10 jours

Nombre moyen de messages sur 10 jours : 1 000 / 365 × 10 = 27,4 messages.

Une seule partition contient donc en moyenne 27 messages.

Pour récupérer 50 messages, il faut consulter deux partitions de 27 messages.

Option C : fenêtre d'un jour

Nombre moyen de messages par jour : 1 000 / 365 = 2,74 messages.

Pour récupérer 50 messages : 50 / 2,74 = 18,25 jours.

Il faut donc consulter environ 19 partitions par jours pour obtenir 50 messages.

Je choisirai la B car elle permet de moins consultés de partitions que l'option C.

Existe-t-il un N qui est bon pour les deux salons :

Pour le grand salon, une fenêtre de 10 jours représente environ 82,2 Mo sur l'exercice précèdent, ce qui reste sous la limite de 100 Mo. Pour le petit salon, une fenêtre de 10 jours contient environ 27 messages. Il faut donc consulter deux partitions pour récupérer 50 messages. On pourrait augmenter la fenêtre pour réduire le nombre de partitions consultées dans le petit salon. Cependant, cela augmenterait la taille des partitions du grand salon. Donc cela dépasserait la limite de 100 Mo.

Pour le compromis, il est assumé que Discord utilise une fenêtre d'environ 10 jours. La raison la plus probable est que cela permet de garder les partitions sous la limite de 100 Mo pour les salons très actifs tout en limitant le nombre de partitions à consulter sur les petits salons.

Puis, pour éviter les lectures inutiles, il est dit que sur Discord qu'ils ont mis en place un suivi des fenêtres vides après avoir rencontré des problèmes liés aux suppressions de messages. C'est pour surement suivre les fenêtres qui contiennent réellement des messages. Lorsqu'une fenêtre est vide, le système peut éviter de l'interroger.

Pourquoi utiliser « message_id « plutôt que l'horodatage comme clé de tri :

C'est parce que Discord utilise des identifiants de messages appelés Snowflakes. Ces identifiants sont des entiers de 64 bits qui encodent notamment l'horodatage de création du message. Ils peuvent donc être triés dans l'ordre chronologique. De plus, le dossier

explique que « created_at » a été écarté, car deux messages peuvent avoir exactement le même horodatage. Avec « message_id », chaque message possède un identifiant unique qui permet de les ordonner sans ambiguïté.

3d : Modes de défaillance

Incident 1 : les NULL écrits

Problème : Dans Cassandra, écrire une valeur NULL peut créer une tombstone, c'est-à-dire un marqueur de suppression. Discord écrivait environ 12 valeurs NULL par message. Ce qui provoque la création d'un grand nombre de tombstones inutiles.

Mécanisme en cause : Le fonctionnement des tombstones dans Cassandra. Ces marqueurs doivent être conservés puis nettoyés par les mécanismes de compaction.

Comment prévoir le problème : Il fallait identifier que les champs facultatifs ne devaient pas être écrits lorsqu'ils n'avaient aucune valeur.

Solution et garde-fou :

- Ne pas écrire les champs dont la valeur est NULL.
- Surveiller le nombre de tombstones créées.
- Surveiller les opérations de compaction.

Correction appliquée par Discord : on ne stocke que les valeurs qui sont non nulles.

Incident 2 : le salon avec des millions de suppressions

Problème : Un salon contenait des millions de messages supprimés, mais seulement un message encore présent. Lorsqu'un utilisateur consultait ce salon, Cassandra devait parcourir des millions de tombstones. En conséquence, cela provoquait des temps de lecture pouvant atteindre environ 20 secondes et des pauses système de 10 secondes.

Mécanisme en cause : Les tombstones générées par les suppressions de messages et leur nettoyage tardif.

Comment prévoir le problème : Il fallait anticiper le coût des suppressions massives et surveiller les partitions contenant beaucoup de tombstones.

Solution et garde-fou :

- Réduire la durée de conservation des tombstones.
- Mettre en place des réparations nocturnes.
- Surveiller les fenêtres vides pour éviter de les interroger inutilement.

Correction appliquée par Discord : la durée de vie des tombstones a été réduite de 10 à 2 jours.

Incident 3 : la partition chaude

Problème : Un grand serveur génère beaucoup plus de trafic qu'un petit serveur. Donc, une même partition peut donc recevoir énormément de lectures et d'écritures. Cela provoque une surcharge des nœuds qui hébergent cette partition et la latence augmente alors pour toutes les requêtes traitées par ces mêmes nœuds.

Mécanisme en cause : La répartition inégale du trafic entre les partitions et les nœuds. Aussi, les lectures sont également plus coûteuses que les écritures dans Cassandra, notamment parce qu'elles peuvent nécessiter de consulter plusieurs SS Tables.

Comment prévoir le problème : Il fallait anticiper les différences de trafic entre les salons et surveiller les partitions les plus sollicitées.

Solution et garde-fou :

- Surveiller les partitions les plus utilisées.
- Éviter de concentrer trop de trafic sur une seule partition.
- Surveiller la latence des lectures et des écritures.
- Suivre le retard de compaction.

Incident 4 : les pauses du ramasse-miettes

Problème : Cassandra fonctionne avec la JVM de Java. Le ramasse-miettes doit régulièrement libérer la mémoire utilisée par les objets qui ne servent plus. Certaines opérations provoquaient des pauses suffisamment longues pour nécessiter le redémarrage d'un nœud par un opérateur.

Mécanisme en cause : Les pauses du ramasse-miettes de la JVM.

Comment prévoir le problème : Il fallait monitorer la mémoire utilisée par les nœuds et mesurer la durée des pauses du ramasse-miettes.

Solution et garde-fou :

- Régler correctement la JVM.
- Surveiller les pauses du ramasse-miettes et la mémoire disponible sur chaque nœud.
- Mettre en place des alertes lorsque les pauses deviennent trop longues.

Phase 4 : Concevoir l'architecture cible.

4a : carte des besoins

Besoin	Moteur retenu	Paradigme	Contrainte décisive	Source de vérité	Comment la donnée y arrive	Ce qu'on accepte de perdre
Comptes utilisateurs	PostgreSQL	Relationnel	Le courriel doit être unique avec intégrité des comptes	Oui	Écriture directe depuis l'application	Pas de disponibilité hors ligne
Serveurs et appartenances	PostgreSQL	Relationnel	Le propriétaire doit être membre et un rôle par appartenance	Oui	Écriture directe et transactionnelle	Pas de duplication des appartenances
Messages	ScyllaDB	Colonnes larges, partitionnement distribué	Afficher les 50 derniers messages. Partitions limitées à environ 100 Mo	Oui, pour le contenu des messages	Écriture directe dans ScyllaDB	Jointures et requêtes ad hoc
Pièces jointes	PostgreSQL et stockage de fichiers	Relationnel et stockage objet	Les formes des pièces jointes sont variables : image, fichier, lien	PostgreSQL pour les métadonnées et stockage de fichiers pour les contenus binaires	Métadonnées en écriture directe et fichier déposé dans le stockage	Recherche complexe dans le contenu des fichiers
Vérification du droit d'écrire	PostgreSQL	Relationnel	L'auteur doit être membre du serveur du salon	Oui	Lecture directe de l'appartenance avant l'envoi	Une vérification supplémentaire avant chaque envoi
Présence en ligne	Redis	Clé-valeur avec	Une présence expire après 5	Oui, pour l'état temporaire	Mise à jour à chaque signal de vie et délai	Historique détaillé des présences

Besoin	Moteur retenu	Paradigme	Contrainte décisive	Source de vérité	Comment la donnée y arrive	Ce qu'on accepte de perdre
		expiration TTL	minutes sans activité		de 300 secondes	
Amitiés et suggestions	PostgreSQL	Relationnel, parcours de relations	Amitié symétrique et une seule relation par paire	Oui	Écriture directe et requêtes avec jointures	Suggestions potentiellement moins rapides à grande échelle
Tableau de bord d'un serveur	PostgreSQL	Agrégats pré-calculés	Nombre de messages et d'auteurs distincts par jour et mise à jour quotidienne suffisante	Non : données calculées à partir des messages	Traitement quotidien à partir des messages	Fraîcheur immédiate des statistiques

4b : Schéma de l'architecture cible

Schéma :

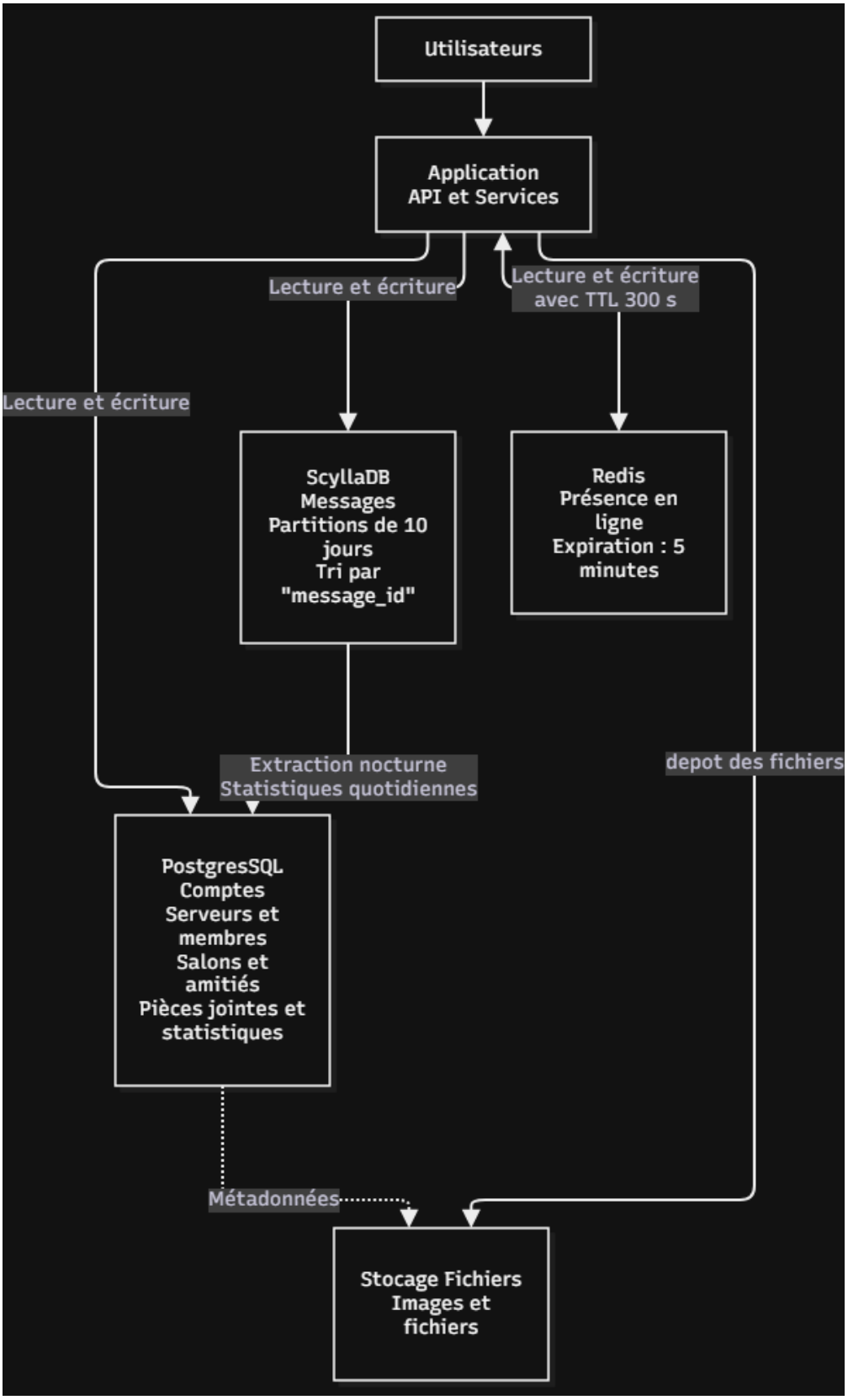


Diagramme de flux :

Flux	Mode	Fraîcheur attendue	Fonction
Application → PostgreSQL	Écriture directe	Immédiatement	Création des comptes, serveurs, appartenances et amitiés
Application → PostgreSQL	Lecture directe	Immédiatement	Vérification des droits d'écriture
Application → ScyllaDB	Écriture directe	Immédiate après confirmation	Enregistrement des messages
Application ← ScyllaDB	Lecture directe	Immédiatement	Récupération des 50 derniers messages
Application ↔ Redis	Mise à jour directe	Immédiatement	Gestion de la présence en ligne
Application → stockage de fichiers	Écriture directe	À l'envoi du fichier	Stockage des pièces jointes
Application → PostgreSQL	Écriture directe	À l'envoi du fichier	Enregistrement des métadonnées et du chemin
ScyllaDB → PostgreSQL	Traitement nocturne	Jusqu'à 24 heures de retard	Calcul des statistiques quotidiennes

4c : Trois décisions d'architecture

Décision 1 : utiliser ScyllaDB pour stocker les messages

Contexte : L'écran principal affiche les 50 derniers messages d'un salon. Cette requête est la plus fréquente de la plateforme. L'historique peut devenir très volumineux et les lectures sont parfois aléatoires.

Décision retenue : utiliser ScyllaDB avec une clé de partition composée du salon et d'une fenêtre temporelle de 10 jours :

Clé de partition : (id_salon, bucket_10_jours)

Clé de tri : message_id

L'application interroge les partitions de la plus récente à la plus ancienne jusqu'à obtenir 50 messages.

Options écartées :

- PostgreSQL seul : les messages et leurs index peuvent devenir trop volumineux pour une architecture relationnelle simple.
- MongoDB : le retour d'expérience de Discord montre des lectures imprévisibles lorsque les données ne tiennent plus en mémoire.
- Une partition unique par salon : risque de dépasser la limite d'environ 100 Mo.
- Une partition par jour : trop de partitions à consulter pour les salons peu actifs.

Conséquences :

- Meilleure répartition de l'historique.
- Lecture organisée autour de la requête principale.
- Nécessité de gérer les fenêtres vides.
- Complexité supplémentaire pour les suppressions et la gestion des tombstones.

Ce qui nous ferait changer d'avis : Si les mesures du jour 2 montrent que le p99 de lecture des 50 derniers messages dépasse régulièrement 100 ms, ou que les partitions de 10 jours dépassent fréquemment 100 Mo, nous réexaminerons la stratégie de partitionnement.

Décision 2 : utiliser PostgreSQL pour les données relationnelles

Contexte :

La plateforme possède des relations nombreuses : utilisateurs, serveurs, appartenances, rôles, amitiés et salons. Plusieurs règles métier doivent être garanties, notamment

l'unicité du courriel, l'unicité du nom d'un salon dans un serveur et la cohérence des appartenances.

Décision retenue :

Utiliser PostgreSQL pour les données relationnelles et transactionnelles.

Les tables principales sont :

- utilisateur
- serveur
- appartenance
- salon
- amitie
- piece_jointe
- statistique_journaliere

Les contraintes d'intégrité sont assurées autant que possible par les clés étrangères, les contraintes « unique », les « check » et les transactions.

Options écartées :

- ScyllaDB pour ces données : les jointures et les contraintes relationnelles sont moins adaptées à ce moteur.
- Redis : il ne convient pas comme stockage principal de données métiers durables.
- MongoDB : les relations et contraintes du modèle seraient plus complexes à garantir.

Conséquences :

- Bonne cohérence des données métier.
- Transactions pour les opérations impliquant plusieurs tables.
- Requêtes relationnelles adaptées aux suggestions d'amis et de serveurs.
- Nécessité de surveiller les performances lorsque le nombre de relations augmente.

Ce qui nous ferait changer d'avis :

Si les requêtes relationnelles deviennent trop lentes malgré les index et l'optimisation, ou si la volumétrie dépasse les capacités de l'instance choisie, nous envisagerons une évolution de l'architecture.

Décision 3 : Utiliser Redis pour la présence en ligne

Contexte :

Le système doit afficher en permanence les utilisateurs en ligne. Une présence expire automatiquement après cinq minutes sans signal de vie.

Il s'agit d'une donnée temporaire qui ne nécessite pas un historique durable.

Décision retenue :

Utiliser Redis avec une clé par utilisateur et une expiration de 300 secondes.

Par Exemple :

presence : utilisateur_1 = online

TTL = 300 secondes

À chaque signal de vie, l'application actualise la valeur et réinitialise son expiration.

Options écartées :

- PostgreSQL : il faudrait gérer régulièrement la suppression des présences expirées.
- ScyllaDB : un stockage distribué durable est peu utile pour une donnée aussi temporaire.
- Une présence sans expiration : risque de conserver des utilisateurs affichés en ligne alors qu'ils sont déconnectés.

Conséquences :

- Expiration automatique des présences.
- Lecture rapide des états en ligne.
- Perte possible des présences en cas de redémarrage ou de perte des données Redis.
- Les utilisateurs devront renvoyer un signal de vie pour réapparaître en ligne.

Ce qui nous ferait changer d'avis :

Si Redis devient un point de saturation ou si les lectures de présence dépassent régulièrement 50 ms, nous étudierons une autre stratégie de stockage ou de répartition.

4d : Le risque principal

Scénario de divergence :

Une erreur se produit pendant le traitement nocturne :

- Certains messages ne sont pas pris en compte.
- Le traitement s'interrompt avant la fin.
- Une partie des statistiques est mise à jour, mais pas le reste.

Le tableau de bord peut alors afficher un nombre de messages inférieur au nombre réel.

Qui s'en aperçoit et quand :

L'équipe de développement ou d'exploitation peut s'en apercevoir :

- Lorsqu'un administrateur compare les statistiques à l'activité réelle.
- Lorsqu'un utilisateur signale une incohérence.
- Lors d'un contrôle automatique entre les données sources et les statistiques calculées.

Dommages possibles :

- Tableau de bord incorrect.
- Mauvaise représentation de l'activité du serveur.
- Perte de confiance dans les statistiques.
- Décisions prises à partir de chiffres erronés.

Mécanisme de détection :

Mettre en place un contrôle automatique quotidien :

1. Recalculer les statistiques d'une journée à partir des messages sources.
2. Comparer les résultats avec les valeurs enregistrées dans PostgreSQL.
3. Déclencher une alerte si les valeurs sont différentes.
4. Enregistrer la date et le résultat de chaque contrôle.

Mécanisme de réparation

En cas de divergence :

- Relancer le calcul de la journée concernée.
- Remplacer les statistiques incorrectes par les valeurs recalculées.
- Conserver une trace de la correction.
- Vérifier que le traitement nocturne suivant s'est terminé correctement.

Garde-fou supplémentaire : le calcul doit être idempotent. Cela signifie que relancer deux fois le traitement d'une même journée ne doit pas doubler les statistiques.

4e : La version minimale

Deux moteurs conservés :

Moteur	Besoins couverts	Justification
PostgreSQL	Comptes, serveurs, appartenances, salons, amitiés, pièces jointes, présence et statistiques	Un moteur relationnel unique permet de gérer les contraintes et les données métier
ScyllaDB	Messages et historique des salons	Adapté aux volumes importants et aux lectures chronologiques partitionnées

Comment couvrir les besoins orphelins ?

Présence :

Au lieu d'utiliser Redis, la présence est stockée temporairement dans PostgreSQL. Une colonne « derniere_activite » permet de déterminer si l'utilisateur est encore en ligne.

Un utilisateur est considéré en ligne si :

« heure_actuelle » - « derniere_activite » < 5 minutes

Un index sur « derniere_activite » peut aider les recherches de présence.

Pièces jointes :

Les métadonnées sont conservées dans PostgreSQL. Les fichiers binaires sont stockés dans un espace de fichiers simple, sans ajouter un moteur de base de données supplémentaire.

Tableau de bord :

Les statistiques quotidiennes sont calculées à partir des messages et enregistrées dans PostgreSQL.

Suggestions :

Les suggestions d'amis et de serveurs sont calculées avec les jointures PostgreSQL.

Quand ajouter un troisième moteur :

Signal mesurable	Seuil d'alerte proposé	Moteur à ajouter	Justification
Présence : latence élevée	p99 supérieur à 50 ms de façon régulière	Redis	Décharger PostgreSQL des lectures et écritures fréquentes de présence
Volume des fichiers	Espace disque supérieur à 80 % ou croissance trop importante	Stockage objet dédié	Séparer les fichiers binaires des données relationnelles

Quand ajouter un quatrième moteur :

Signal mesurable	Seuil d'alerte proposé	Moteur à ajouter	Justification
Recherche textuelle	p95 supérieur à 300 ms sur les recherches, malgré l'optimisation	Moteur de recherche spécialisé	Permettre une recherche plein texte sans surcharger ScyllaDB

4f : Plan de vérification pour le jour 2

N°	Hypothèse à tester	Explication du mécanisme	Si l'hypothèse est fausse, nous conclurons que...
1	ScyllaDB sera environ 2 fois plus rapide que PostgreSQL pour afficher les 50 derniers messages d'un salon.	ScyllaDB utilise une clé de partition et une clé de tri adaptées à cette lecture chronologique.	PostgreSQL est suffisamment performant pour cette charge, ou le coût d'accès aux partitions ScyllaDB réduit son avantage.
2	ScyllaDB avec des fenêtres de 10 jours sera plus rapide qu'une partition unique par salon pour les salons très actifs, avec un facteur d'environ 2.	Le découpage évite de créer des partitions trop volumineuses.	Le partitionnement en fenêtres ajoute trop de requêtes ou le volume testé ne suffit pas à révéler un problème de taille.
3	Probablement que Redis sera environ 3 fois plus rapide que PostgreSQL pour lire et mettre à jour la présence d'un utilisateur.	Redis est conçu pour les accès clé-valeur en mémoire et propose une expiration TTL native.	PostgreSQL suffit pour cette charge ou les performances de Redis ne justifient pas un moteur supplémentaire.
4	Je pense que PostgreSQL sera environ 2 fois plus rapide que ScyllaDB pour vérifier si un utilisateur est membre d'un serveur.	PostgreSQL utilise une relation avec clés étrangères et un index sur les identifiants utilisateur et serveur.	La vérification dans ScyllaDB est aussi performante, ou le modèle relationnel PostgreSQL n'est pas suffisamment optimisé.
5	Peut-être qu'un tableau de bord basé sur des statistiques précalculées dans PostgreSQL sera environ 5 fois plus rapide qu'un calcul direct à partir des messages dans ScyllaDB.	Les statistiques sont déjà agrégées par serveur et par jour, ce qui évite de parcourir l'historique complet.	Le volume de messages est trop faible pour justifier une pré-agrégation, ou le traitement et la lecture des statistiques ne sont pas suffisamment optimisés.